MongoDB คืออะไร?

ความรู้พื้นฐานและการนำไปใช้งานจริง



Woottikarn Hongwiengchan

MongoDB คืออะไร?

- MongoDB เป็นฐานข้อมูลแบบ **NoSQL** (Non-relational) ที่ได้รับความนิยมสูง
- เก็บข้อมูลในรูปแบบ "เอกสาร" (Documents) ที่ยืดหยุ่นคล้าย JSON
- เรียกว่า BSON (Binary JSON) ซึ่งมีประสิทธิภาพในการจัดเก็บและค้นหา
- ถูกออกแบบมาเพื่อความยืดหยุ่น (Flexibility) และการขยายระบบ (Scalability)



SQL (Relational) vs. NoSQL (MongoDB)

ฐานข้อมูล SQL (เช่น MySQL)

- โครงสร้างแบบตาราง (Tables)
- Schema ที่ตายตัว (Fixed Schema)
- เหมาะสำหรับข้อมูลที่มีความสัมพันธ์ชัดเจน

ฐานข้อมูล NoSQL (MongoDB)

- โครงสร้างแบบเอกสาร (Documents)
- Schema ที่ยืดหยุ่น (Dynamic Schema)
- เหมาะสำหรับข้อมูลที่ไม่มีโครงสร้างตายตัว

แนวคิดหลักของ MongoDB (Core Concepts)



Database

แหล่งเก็บข้อมูลหลัก (เหมือนใน SQL)



Collection

กลุ่มของเอกสาร (คล้ายกับ Table ใน SQL)



Document

ข้อมูล 1 รายการ มีลักษณะเป็น BSON (คล้ายกับ Row ใน SQL)

ตัวอย่าง Document (BSON)

{ "username": "johndoe", "email": "john@example.com", "status": "active", "orders": [{ "item": "laptop", "qty": 1 }, { "item": "mouse", "qty": 1 }] }

ข้อมูลถูกเก็บในรูปแบบ Key-Value ที่ยืดหยุ่น สามารถซ้อนกันได้

ความแตกต่างระหว่าง SQL กับ MongoDB

หัวข้อ	SQL (ฐานข้อมูลเชิงสัมพันธ์)	MongoDB (ฐานข้อมูลเอกสาร)
ฐานข้อมูล (Database)	ใช้เก็บข้อมูลในรูปแบบตาราง	ใช้เก็บข้อมูลในรูปแบบคอลเลกชัน (Collection)
ตาราง (Table)	โครงสร้างหลักที่เก็บข้อมูล	คอลเลกชัน (Collection) คือชุดของเอกสาร
แถว (Row)	แถวหนึ่งคือระเบียนข้อมูล (Record)	เอกสาร (Document) หรือ BSON Document
คอลัมน์ (Column)	แต่ละคอลัมน์เก็บค่าของฟิลด์หนึ่ง ๆ	ฟิลด์ (Field) ในเอกสาร
ดัชนี (Index)	ใช้เพิ่มความเร็วในการค้นหา	มีดัชนีเหมือนกัน ใช้เพิ่มประสิทธิภาพการค้นหา
การเชื่อมตาราง (Join)	ใช้คำสั่ง JOIN เพื่อรวมข้อมูลจากหลายตาราง	ใช้ “เอกสารฝัง (Embedded Document)” หรือ “การเชื่อมโยง (Linking)” แทน
คีย์หลัก (Primary Key)	ระบุคอลัมน์ที่เป็นเอกลักษณ์ได้เอง	มี _id เป็นคีย์หลักอัตโนมัติในทุกเอกสาร
การรวมข้อมูล (Aggregation)	ใช้ฟังก์ชันเช่น GROUP BY	ใช้ “Aggregation Pipeline” สำหรับการรวมและประมวลผลข้อมูล

คุณสมบัติเด่นของ MongoDB (Key Features)

- **Horizontal Scaling (Sharding):** ขยายระบบได้ง่ายในแนวนอนเพื่อรองรับข้อมูลจำนวนมหาศาล
- **High Availability (Replica Sets):** ทำสำเนาข้อมูล (Replica) อัตโนมัติ เพื่อป้องกันข้อมูลสูญหาย
- **Powerful Queries:** มีระบบ Query Language (MQL) และ Aggregation Pipeline ที่ทรงพลัง
- **Flexible Schema:** ไม่ต้องกำหนดโครงสร้างข้อมูลตายตัว เพิ่ม-ลด field ได้ง่าย

ตัวอย่างการนำไปใช้งานจริง (Use Cases)



E-Commerce

ระบบ E-Commerce (เก็บข้อมูลสินค้า, ตะกร้าสินค้า)



Content Management (CMS)

ระบบจัดการเนื้อหา (เก็บ Blog, บทความ)



Internet of Things (IoT)

แอปพลิเคชัน IoT (เก็บข้อมูลจาก Sensors จำนวนมาก)

MongoDB แพลตฟอร์ม

แพลตฟอร์ม	ประเภท	จุดเด่น	เหมาะกับ
MongoDB Atlas	Cloud Service	ใช้ได้ทุกที่, มีเครื่องมือครบ	ทุกระดับ
MongoDB Compass	GUI Desktop	ใช้งานง่าย ดูข้อมูลได้ทันที	มือใหม่ / Analyst
Mongo Shell (mongosh)	CLI	พิมพ์คำสั่งได้อิสระ	Developer / DBA
VS Code Extension	IDE Plugin	ใช้ได้ใน VS Code โดยตรง	Developer
MongoDB Charts	Web Visualization	สร้างกราฟและ Dashboard	Analyst
MongoDB Realm	Serverless / Mobile	เชื่อมต่อมือถือ & backend	Mobile Dev
Atlas CLI	Command-line admin	บริหารจัดการ Atlas	DevOps / Admin

MongoDB Compass (GUI Client)

- เครื่องมือแบบ Desktop GUI สำหรับดูและจัดการข้อมูล
- ใช้งานง่าย ไม่ต้องเขียนคำสั่งเยอะ
- ใช้สำหรับเชื่อมต่อกับ MongoDB ทั้ง local และ Atlas
- เหมาะกับ: ผู้เริ่มต้น, นักวิเคราะห์ข้อมูล, Developer ที่อยากดูข้อมูลแบบตาราง
- ลิงค์ : https://www.mongodb.com/try/download/community



MongoDB Query Language (MQL)

คำสั่งในการจัดการข้อมูลใน MongoDB พื้นฐาน

ภาพรวมของ MQL

- MQL (MongoDB Query Language) คือภาษาสำหรับสืบค้นและจัดการข้อมูลใน MongoDB
- ใช้รูปแบบคำสั่งแบบ JSON
- ใช้กับคำสั่งพื้นฐาน เช่น find(), insert(), update(), delete()
- สนับสนุนการกรอง (filter), การจัดเรียง (sort), การรวมข้อมูล (aggregate)

พื้นฐาน: การค้นหาข้อมูล (find)

แสดงข้อมูลทั้งหมด: db.customer.find()

แสดงข้อมูลให้อ่านง่าย(สวยงาม): db.customer.find().pretty()

ค้นหาด้วยเงื่อนไข: db.customer.find({Country: 'Austria'})

แสดงเฉพาะบาง field: db.customer.find({}, {City: 1, Country: 1, _id: 0})

*customer ชื่อ Collection (เหมือนตารางใน sql)

การเปรียบเทียบระหว่าง MQL (MongoDB Query Language) กับ SQL

SQL	MongoDB (MQL)	ความหมาย
SELECT * FROM customers	db.customers.find()	แสดงข้อมูลทั้งหมด
SELECT name, city FROM customers	db.customers.find({}, { name: 1, city: 1 })	แสดงเฉพาะฟิลด์ name, city
SELECT * FROM customers WHERE city = 'Berlin'	db.customers.find({ city: "Berlin" })	กรองข้อมูลตามเงื่อนไข
SELECT * FROM customers WHERE age > 30	db.customers.find({ age: { $gt: 30 } })	เงื่อนไขมากกว่า (greater than)
SELECT * FROM customers ORDER BY age DESC	db.customers.find().sort({ age: -1 })	เรียงลำดับจากมากไปน้อย
SELECT COUNT(*) FROM customers	db.customers.countDocuments()	นับจำนวนแถว
SELECT * FROM customers LIMIT 5	db.customers.find().limit(5)	จำกัดจำนวนผลลัพธ์

การใช้ : 1 และ : 0

รูปแบบ	ความหมาย	ตัวอย่าง
"ชื่อฟิลด์": 1	✓ แสดงฟิลด์นี้	"CustomerID": 1
"ชื่อฟิลด์": 0	✗ ไม่แสดงฟิลด์นี้	"City": 0

Question ?

- ถ้าจะให้แสดงข้อมูลโดยใช้เงื่อนไข <, <= , == , >= จะเขียน Query อย่างไร
- ถ้าจะ Sort ข้อมูลจากน้อยไปมากจะเขียน Query อย่างไร

ตารางสรุป Comparison Operators ใน MongoDB

Operator	ความหมาย (ไทย)	ตัวอย่างคำสั่ง	อธิบายผลลัพธ์
$eq	เท่ากับ (=)	db.customers.find({ age: { $eq: 30 } })	แสดง document ที่มี age เท่ากับ 30
$ne	ไม่เท่ากับ (!=)	db.customers.find({ age: { $ne: 30 } })	แสดง document ที่มี age ไม่เท่ากับ 30
$gt	มากกว่า (>)	db.customers.find({ age: { $gt: 30 } })	แสดง document ที่มี age มากกว่า 30
$gte	มากกว่าหรือเท่ากับ (≥)	db.customers.find({ age: { $gte: 30 } })	แสดง document ที่มี age มากกว่าหรือเท่ากับ 30
$lt	น้อยกว่า (<)	db.customers.find({ age: { $lt: 30 } })	แสดง document ที่มี age น้อยกว่า 30
$lte	น้อยกว่าหรือเท่ากับ(≤)	db.customers.find({ age: { $lte: 30 } })	แสดง document ที่มี age น้อยกว่าหรือเท่ากับ 30

การจัดเรียงและจำกัดผลลัพธ์

เรียงลำดับ:

```
db.customer.find().sort({City: 1})  // จาก A-Z
db.customer.find().sort({City: -1}) // จาก Z-A
```

จำกัดจำนวนผลลัพธ์:

```
db.customer.find().limit(5)
```

ข้ามเอกสาร N รายการแรก แล้วค่อยแสดงผลต่อจากนั้น:

```
db.customer.find().skip(5)
```

การใช้เงื่อนไขหลายตัวพร้อมกัน

เงื่อนไข	ตัวอย่างคำสั่ง	ผลลัพธ์
ช่วงค่า (Between)	db.customers.find({ age: { $gte: 20, $lte: 40 } })	หาอายุระหว่าง 20–40
ไม่อยู่ในช่วง	db.customers.find({ age: { $not: { $gte: 20, $lte: 40 } } })	หาอายุที่ไม่อยู่ในช่วง 20–40

ตัวอย่างใช้งานคำสั่ง skip

ลำดับ	ชื่อ (Name)
1	A
2	B
3	C
4	D
5	E
6	F
7	G
8	H

```
db.customer.find().skip(5)
```

ตัวอย่างผลลัพธ์

ลำดับ	ชื่อ (Name)
6	F
7	G
8	H

ตัวอย่างคำสั่ง (Update & Delete)

Update (แก้ไข)

อัปเดต Document ที่ 'name' คือ "Alice" โดยตั้งค่า 'age' เป็น 30

```
db.users.updateOne({ "name": "Alice" }, { $set: { "age": 30 } })
```

Delete (ลบ)

ลบ Document แรกที่ 'name' คือ "Alice"

```
db.users.deleteOne({ "name": "Alice" })
```

Aggregation Framework

คำสั่งสำหรับรวมและคำนวณข้อมูลหลายชั้น

แนวคิดพื้นฐานของ Aggregation

- ข้อมูลจะ ไหลผ่านหลายขั้นตอน (stages)
- แต่ละ stage จะแปลงหรือกรองข้อมูล ก่อนส่งต่อไปยังขั้นตอนถัดไป
- คล้าย ๆ กับการใช้ GROUP BY, SUM(), AVG() ใน SQL แต่ยืดหยุ่นมากกว่า

Collection → [Stage1] → [Stage2] → [Stage3] → ผลลัพธ์

โครงสร้างพื้นฐานของคำสั่ง

```
db.collection.aggregate([
  { <stage1> },
  { <stage2> },
  ...
])
```

ตัวอย่างการใช้งาน : Aggregation

```
db.customers.aggregate([
  { $match: { Country: "UK" } },     // เลือกเฉพาะลูกค้าใน UK
  { $group: { _id: "$City", total: { $sum: 1 } } }  // นับจำนวนลูกค้าในแต่ละเมือง
])
```

รูปแบบ	ความหมาย
{ $sum: 1 }	นับจำนวน (count)
{ $sum: "$field" }	รวมค่าของ field นั้น
{ $sum: { $cond: [...] } }	รวมแบบมีเงื่อนไข (conditional sum)

ขั้นตอน (Stages) ที่นิยมใช้บ่อย

Stage	หน้าที่	ตัวอย่าง
$match	กรองข้อมูล (เหมือน WHERE)	{ $match: { age: { $gte: 30 } } }
$project	เลือกเฉพาะ field ที่ต้องการแสดง	{ $project: { name: 1, age: 1 } }
$group	รวมกลุ่มข้อมูล (เหมือน GROUP BY)	{ $group: { _id: "$country", total: { $sum: 1 } } }
$sort	เรียงลำดับข้อมูล	{ $sort: { age: -1 } }
$limit	จำกัดจำนวนผลลัพธ์	{ $limit: 5 }
$skip	ข้ามข้อมูลบางส่วน	{ $skip: 10 }
$count	นับจำนวนเอกสาร	{ $count: "total_customers" }
$lookup	join กับอีก collection (เหมือน SQL JOIN)	{ $lookup: { from: "orders", localField: "CustomerID", foreignField: "CustomerID", as: "orders" } }
$unwind	แยก array ออกเป็นหลายแถว	{ $unwind: "$orders" }
$addFields	เพิ่มฟิลด์ใหม่ (คำนวณหรือแปลงข้อมูล)	{ $addFields: { ageGroup: { $cond: [{ $gte: ["$age", 30] }, "Adult", "Young"] } } }

ตัวอย่างแต่ละคำสั่ง

สมมติคุณมี **collection orders:**

```
[
  { "cust": "A", "amount": 100, "city": "Bangkok" },
  { "cust": "B", "amount": 200, "city": "Chiang Mai" },
  { "cust": "A", "amount": 150, "city": "Bangkok" }
]
```

ตัวอย่างการใช้งาน

เราต้องการ ยอดขายรวมของลูกค้าแต่ละคนโดยเรียงข้อมูลจากยอดขายมากไปน้อย

```
db.orders.aggregate([
  { $group: { _id: "$cust", totalAmount: { $sum: "$amount" } } },
  { $sort: { totalAmount: -1 } }
])
```

```
{
  _id: ObjectId('690f396dbb0ee27b4a3800a2'),
  CustomerID: 'EASTC',
  CompanyName: 'Eastern Connection',
  ContactName: 'Vanhein',
  ContactTitle: 'Sales Agent',
  Address: '35 King George',
  City: 'London',
  Region: null,
  PostalCode: 'WX3 6FW',
  Country: 'UK',
  Phone: '(171) 555-0297',
  Fax: '(171) 555-3373',
  Age: 25,
  Orders: [
    { OrderID: 1, Total: 120 },
    { OrderID: 2, Total: 200 },
    { OrderID: 3, Total: 80 }
  ]
}
```

การใช้ $unwind

แสดงข้อมูล Order ของแต่ละลูกค้า โดยแยกออกมาเป็นแถว ๆ

```
db.customer.aggregate([
  { $unwind: "$Orders" },
  { $project: { CustomerID: 1, ContactName:
1, "Orders.OrderID": 1, "Orders.Total": 1 } }
])
```

```
{
  _id: ObjectId('690f396dbb0ee27b4a3800a2'),
  CustomerID: 'EASTC',
  CompanyName: 'Eastern Connection',
  ContactName: 'Vanhein',
  ContactTitle: 'Sales Agent',
  Address: '35 King George',
  City: 'London',
  Region: null,
  PostalCode: 'WX3 6FW',
  Country: 'UK',
  Phone: '(171) 555-0297',
  Fax: '(171) 555-3373',
  Age: 25,
  Orders: [
    { OrderID: 1, Total: 120 },
    { OrderID: 2, Total: 200 },
    { OrderID: 3, Total: 80 }
  ]
}
```

การใช้ $unwind

จงหายอดรวมการสั่งซื้อ (Total) ของลูกค้าแต่ละคน

```
db.customer.aggregate([
  { $unwind: "$Orders" },
  { $group: { _id: "$CustomerID", totalSpent:
{ $sum: "$Orders.Total" } } }
])
```

```
{
  _id: ObjectId('690f396dbb0ee27b4a3800a2'),
  CustomerID: 'EASTC',
  CompanyName: 'Eastern Connection',
  ContactName: 'Vanhein',
  ContactTitle: 'Sales Agent',
  Address: '35 King George',
  City: 'London',
  Region: null,
  PostalCode: 'WX3 6FW',
  Country: 'UK',
  Phone: '(171) 555-0297',
  Fax: '(171) 555-3373',
  Age: 25,
  Orders: [
    { OrderID: 1, Total: 120 },
    { OrderID: 2, Total: 200 },
    { OrderID: 3, Total: 80 }
  ]
}
```

การใช้ $unwind

จงเพิ่มฟิลด์ใหม่ชื่อ AverageOrder ที่เป็นค่าเฉลี่ยของยอดสั่งซื้อแต่ละคน

```
db.customer.aggregate([
  { $unwind: "$Orders" },
  { $group: {
      _id: "$CustomerID",
      totalSpent: { $sum: "$Orders.Total" },
      orderCount: { $sum: 1 }
  }},
  { $addFields: {
      AverageOrder: { $divide: ["$totalSpent",
"$orderCount"] }
  }}
])
```

รวมคำสั่ง “กลุ่มคำนวณ” (Accumulator Operators)

ประเภท	ก่อน **$group**	หลัง **$group**
หน่วยข้อมูล	document เดี่ยว	กลุ่มของ document
ใช้ operator	$add, $subtract, $multiply, $divide, $mod, $ceil, $floor, $round	$sum, $avg, $min, $max, $first, $last, $push, $addToSet
ใช้ $project, $addFields	✓ ได้	✓ ได้ (ใช้กับผลลัพธ์รวม)
ใช้ $match	✓ ใช้กรองเอกสาร	✓ ใช้กรองกลุ่ม (หลัง $group)

ตัวอย่างการใช้ $lookup

```
{
  _id: ObjectId("690f396dbb0ee27b4a3800a2"),
  CustomerID: "EASTC",
  CompanyName: "Eastern Connection",
  ContactName: "Vanhein",
  ContactTitle: "Sales Agent",
  City: "London",
  Country: "UK",
  Age: 25
}
```

ตาราง Customer

ตาราง Order

```
[
  { OrderID: 1, CustomerID: "EASTC", OrderDate: "2024-09-01", Total: 120 },
  { OrderID: 2, CustomerID: "EASTC", OrderDate: "2024-09-12", Total: 200 },
  { OrderID: 3, CustomerID: "EASTC", OrderDate: "2024-10-03", Total: 80 },
  { OrderID: 4, CustomerID: "NORTS", OrderDate: "2024-11-01", Total: 150 }
]
```

ตัวอย่างการใช้ $lookup

ตัวอย่าง Query

```
db.customers.aggregate([
  {
    $lookup: {
      from: "orders",          // ชื่อ collection ที่จะ join
      localField: "CustomerID", // ฟิลด์ใน customers
      foreignField: "CustomerID", // ฟิลด์ใน orders
      as: "OrderList"           // ชื่อฟิลด์ใหม่ที่เก็บข้อมูล orders ที่เชื่อมโยง
    }
  }
])
```

ตัวอย่างการใช้ $lookup

ผลลัพธ์จะได้แบบนี้

```
[
  {
    _id: ObjectId("690f396dbb0ee27b4a3800a2"),
    CustomerID: "EASTC",
    CompanyName: "Eastern Connection",
    ContactName: "Vanhein",
    City: "London",
    Country: "UK",
    Age: 25,
    OrderList: [
      { OrderID: 1, CustomerID: "EASTC", Total: 120 },
      { OrderID: 2, CustomerID: "EASTC", Total: 200 },
      { OrderID: 3, CustomerID: "EASTC", Total: 80 }
    ]
  }
]
```

ตัวอย่างการใช้ $lookup

รวมยอดคำสั่งซื้อและยอดซื้อรวมของลูกค้าแต่ละคน

```
db.customers.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "CustomerID",
      foreignField: "CustomerID",
      as: "OrderList"
    }
  },
  { $unwind: "$OrderList" },
  {
    $group: {
      _id: "$CustomerID",
      totalSpent: { $sum: "$OrderList.Total" },
      orderCount: { $sum: 1 }
    }
  }
])
```

ผลลัพธ์

```
 _id: 'EASTC',
totalSpent: 1600,
orderCount: 12
```

พื้นฐานการใช้ Regex ใน MongoDB

รูปแบบ	ความหมาย
{ field: { $regex: /pattern/ } }	ค้นหาค่าที่ตรงกับ pattern
{ field: { $regex: "pattern", $options: "i" } }	ใช้ตัวเลือกเพิ่มเติม เช่นไม่สนตัวพิมพ์เล็ก/ใหญ่

Option	ความหมาย
i	ไม่สนตัวพิมพ์เล็ก/ใหญ่ (case-insensitive)
m	multi-line (จับหลายบรรทัด)
s	dotAll (ให้ . จับ \n ได้ด้วย)
x	ข้ามช่องว่างใน pattern (อ่านง่ายขึ้น)

สรุป Regex Pattern ที่ใช้บ่อย

Pattern	ความหมาย	ตัวอย่าง
^text	เริ่มต้นด้วย text	^Ann → “Ann”, “Anna”
text$	ลงท้ายด้วย text	Devon$ → “Ann Devon”
.	ตัวอักษรใด ๆ 1 ตัว	A.n → “Ann”, “Aen”
.*	ตัวอักษรใด ๆ ไม่จำกัด	A.*n → “Ann”, “Aaron”, “Alyson”
\d	ตัวเลข 0–9	\d+ → “123”
\w	ตัวอักษรหรือเลข	\w+ → “abc123”
[abc]	ตัวอักษรใด ๆ ที่อยู่ในชุด	[Aa]nn → “Ann”, “ann”
[^abc]	ตัวอักษรที่ ไม่ใช่ ในชุด	[^A]nn → “Bnn”, “Cnn”
{n,m}	ซ้ำอย่างน้อย n ครั้ง และไม่เกิน m ครั้ง	\d{3,5} → ตัวเลข 3–5 หลัก

ตัวอย่างการใช้งานจริง

ค้นหาคำที่ "มีคำว่า ann" อยู่ตรงไหนก็ได้

```
db.customers.find({"ContactName" : { $regex: /ann/i } })
```

หรือ

```
db.customers.find({"ContactName" : { $regex: "ann", $options: "i" } })
```

ตัวอย่างการใช้งานจริง

ค้นหาชื่อที่ "ขึ้นต้นด้วย A"

```
db.customers.find({"ContactName" : { $regex: /^A/ } })
```

ค้นหาชื่อที่ "ลงท้ายด้วย n"

```
db.customers.find({"ContactName" : { $regex: /n$/i } })
```

ตัวอย่างการใช้งานจริง

ค้นหาคำที่มีตัวเลขในฟิลด์ (เช่นเบอร์โทร) จับเบอร์โทรรูปแบบ “123-4567”

```
db.customers.find({"Phone" : { $regex: /\d{3}-\d{4}/ } })
```

ค้นหาชื่อที่ "มีคำว่า Ann หรือ Van"

```
db.customers.find({"ContactName" : { $regex: /(Ann|Van)/i } })
```

ตัวอย่างการใช้งานจริง

ค้นหาชื่อที่มีตัวอักษร 3 ตัวพอดี

```
db.customers.find({"ContactName" : { $regex: /^...$/ } })
```

ค้นหาอีเมลที่ลงท้ายด้วย .com

```
db.users.find({"email" : { $regex: /\.com$/i } })
```

ตัวอย่างการใช้งานจริง

ค้นหาข้อความที่มีตัวเลข 2–4 หลักติดกัน

```
db.orders.find({ OrderCode: { $regex: /\d{2,4}/ } })
```

ค้นหาชื่อที่ไม่ขึ้นต้นด้วย A

```
db.customers.find({ ContactName: { $not: /^A/ } })
```